

ARRAYS 3

Don't fear failure
be afraid of not taking a
chance

Car Story 3
@ELLECOOPER.CO

Good
Morning



Topics

01. Merge Interval
02. Insert Interval
03. First Missing Integer

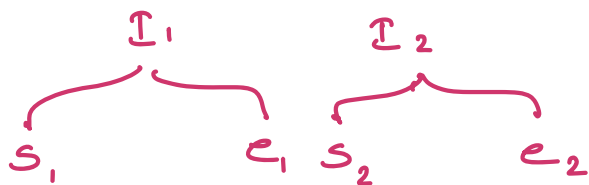
Starting at 7:06 AM

Merged Interval

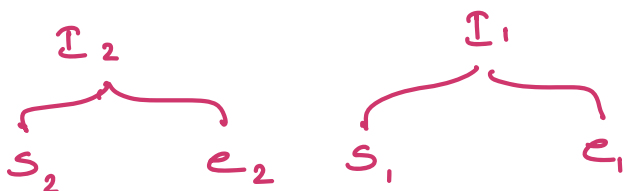
An interval is defined by start & end time.

I_1	I_2		Merged Interval
$[2, 6]$	$[3, 7]$		$\{2, 7\}$
$[2, 8]$	$[4, 6]$		$\{2, 8\}$
$[2, 4]$	$[5, 7]$		No overlapping
$[3, 7]$	$[4, 10]$		$\{3, 10\}$
$[3, 6]$	$[6, 10]$		$\{3, 10\}$
$[2, 5]$	$[8, 10]$		No overlapping
$[5, 8]$	$[1, 3]$		No overlapping
$[3, 9]$	$[1, 4]$		$\{1, 9\}$

* Non overlapping

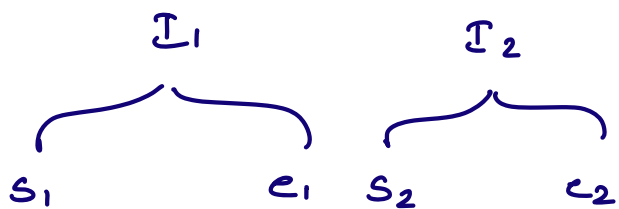


$$\Rightarrow e_1 < s_2$$



$$\Rightarrow e_2 < s_1$$

* Overlapping scenario



Merged Interval

$$\text{ans} = \left\{ \min(s_1, s_2), \max(e_1, e_2) \right\}$$

* Merge Intervals

Q Given a sorted list of overlapping intervals, sorted based on start time, merge all overlapping intervals & return sorted list.

$$\text{Interval}[] = \left\{ [0-2] [1-4] [5,6] \underline{[6,8]} \underline{[7,10]} [8,9] [12,14] \right\}$$

$\uparrow$ start $\uparrow$ end

Interval 1

Interval 2

Ans

$[0-2]$

$[1-4]$

Overlapping

$[0-4]$

$[0-4]$

$[5-6]$

Non-overlapping

$[0-4] [5,6]$

$[5-6]$

$[6-8]$

Overlapping

$[0-4] [5-8]$

$[5-8]$

$[7-10]$

Overlapping

$[0-4] [5-10]$

$[5-10]$

$[8-9]$

Overlapping

$[0-4] [5-10]$

$[5-10]$

$[12-14]$

Non-overlapping

$[0-4] [5-10] [12-14]$

- * Overlapping $\rightarrow$ Merge interval & update last inserted one
- * Non-overlapping $\rightarrow$ Insert new interval

list < Interval > ans;

int st = A[0].start

int end = A[0].end

for (i=1; i < A.length; i++) {

if (A[i].start $\leq$ end) { //merge overlapping

st = min(st, A[i].start);

end = max(end, A[i].end);

}

else {

ans.add(new Interval(st, end));

st = A[i].start;

end = A[i].end;

}

}

ans.add(new Interval(st, end));

TC: $O(n)$

SC: $O(1)$

Interval [] = { (0, 2) (2, 6) (5, 8) (9, 10) (9, 12) (11, 17) }

ans = { (0, 8) (9, 17) }

st = ~~0~~ 9

end = ~~2 6 8 10 12~~ 17

* Insert Interval

Q Given N non-overlapping intervals & they are sorted based on start time. You are given a new interval Merge the new interval & make all the non overlapping intervals

Eg: N = 7

New interval

New intervals after insertion

[1 3]

{1, 3}

[4 7]

{4 7}

[10 14] + {10 22} → {10, 22}

[16 19] + {10, 22} → {10, 22}

[21 24] + {10, 22} → {10, 24}

{10, 24}

{27, 30}

{32, 35}

[27 30]

{10, 24}

[32 35]

* Example 2

New Interval

[1 5]

[8 10]

[11 14] + {12, 22} → {11, 22}

[15 20] + {11, 22} → {11, 22}

[20 24] + {11, 22} → {11, 24}

Ans of non-overlapping
intervals

{1, 5}

{8, 10}

{11, 24}

list <Interval> ans;

for (i=0; i<n; i++){

if (ar[i].end < I.start){

ans.add(ar[i]);

}

else if (ar[i].start > I.end){ // Non-overlapping

ans.add(I);

for (j=i; j<n; j++){

ans.add(ar[j]);

}

return ans;

Ⓘ

Tc: O(n)

Sc: O(1)

// Non overlapping

// Non-overlapping

```

    3
    else {
        I.start = min(ar[i].start, I.start);
        I.end = max(ar[i].end, I.end);
    }
}
3

```

ans.insert(I);

return ans;

8:08 AM → 8:18 AM

*

Q - Find the first missing positive integer / Natural no.

ar[6] = { 3, -1, 1, 2, 7, 0 } Ans = 4

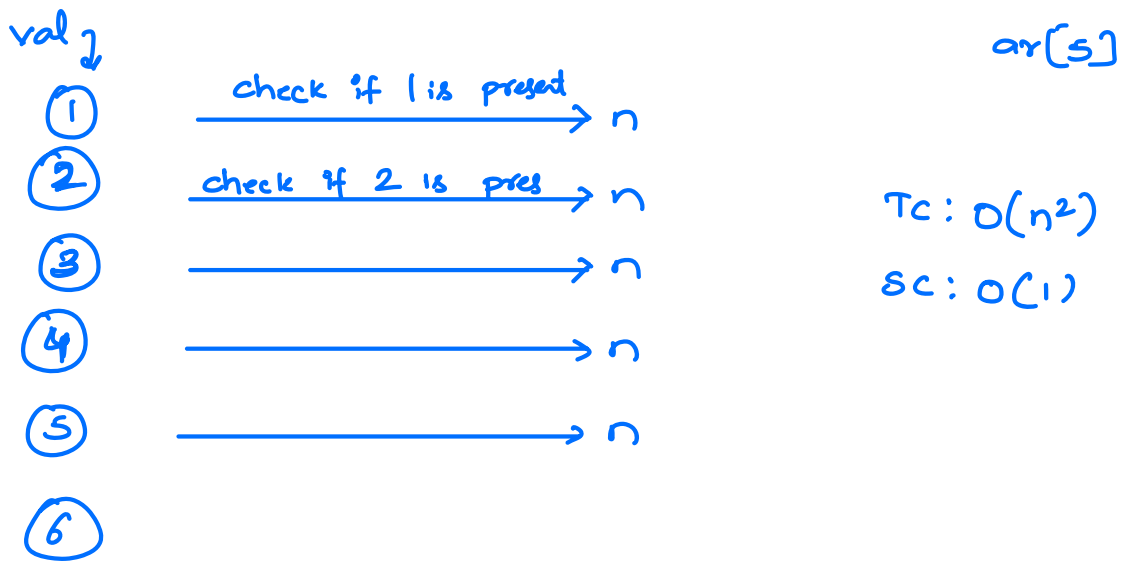
ar[6] = { 1, 0, -5, -6, 4, 2 } Ans = 3

ar[7] = { -9, 2, 6, 4, -8, 1, 3 } Ans = 5

ar[] = { -9, -8, -3 } Ans = 1

ar[] = { 1, 3, 2, 5, 4 } Ans = 6

Brute force $\rightarrow$ Start searching from 1 till $n+1$ in your array



Idea 2 $\rightarrow$ Use HashSet

01. Put all ele in hs $\checkmark$

02. Start from 1 till $n+1$

arr[] = {1, 2, 7, 4}

hs = {1, 2, 4, 7}

for ($i=1$; $i \leq n+1$; $i++$) {

if (hs.contains(i) == false) return i;

}

Tc: $O(n)$

Sc: $O(n)$

Note $\rightarrow$ Can't take extra space

Idea 3 $\rightarrow$ Use sorting

* $arr[] = \{-3, -7, 1, 2, 3, 8, 5\}$

$\downarrow$ sort

$arr[] = \{-7, -3, 1, 2, 3, 5, 8\}$ Ans = 4

① ② ③ 4
x

* $arr[] = \{-3, -7, 1, 2, 3\}$

$\downarrow$ sort

$arr[] = \{-7, -3, 1, 2, 3\}$ Ans = 4

val = 2 3 4

* $arr[] = \{-3, 1, 1, 1, 2, 4\}$ $\Leftarrow$ sort

val = 1 2 3

$\rightarrow$ Arrays.sort(arr):

int val = 1

for (i = 0; i < n; i++) {

 if (arr[i] < 1) continue;

 else {

 if (arr[i] == val) val = val + 1

TC: $O(n \log n)$

SC: depends upon
the Arrays.sort

```
else if (ar[i] == val-1) continue;
```

```
else { break }
```

```
return val;
```

Constraint

→

Tc: $O(n)$

Sc: $O(1)$

Idea 4 → Keep the elements at correct position

ar [] = { $\begin{matrix} 1 \\ 0 \end{matrix}$, $\begin{matrix} 2 \\ 1 \end{matrix}$, $\begin{matrix} 3 \\ 2 \end{matrix}$, $\begin{matrix} 4 \\ 3 \end{matrix}$, $\begin{matrix} 5 \\ 4 \end{matrix}$, $\begin{matrix} 6 \\ 5 \end{matrix}$ }

val

correct idx

1

0

2

1

⋮

$x \longrightarrow \underline{x-1}$ position

ar[s] = { $\begin{matrix} 1 \\ 0 \end{matrix}$, $\begin{matrix} 2 \\ 1 \end{matrix}$, $\begin{matrix} 3 \\ 2 \end{matrix}$, $\begin{matrix} 4 \\ 3 \end{matrix}$, $\begin{matrix} 9 \\ 4 \end{matrix}$ }

Min ans = 1

Max ans = 6

$\left. \begin{array}{l} \text{val} < 1 \\ \text{val} > N \end{array} \right\} \text{do nothing}$

$\text{arr}[] = \left\{ \overset{1}{\cancel{-5}}, -2, \overset{3}{\cancel{5}}, \overset{-5}{\cancel{3}} \right\}$
 0 1 2 3

<u>i</u>	<u>val(arr[i])</u>	
0	-5	{do nothing}
1	-2	{do nothing}
2	1	{swap(arr[0], arr[2]), do nothing}
3	3	{swap(arr[3], arr[2]), do nothing}

Ans = 2

$\text{arr}[] = \left\{ \overset{2}{\cancel{8}}, 1, \overset{4}{\cancel{4}}, 9, \overset{8}{\cancel{2}} \right\}$
 0 1 2 3 4

↑

Ans = 3

val = 4 2

corr_idx = val - 1 ⇒ 3 1

arr[] = { ~~2~~
~~4~~ 2 2 4 -10 7 }
 0 1 2 3 4 5

↑ Ans = 3

$$\sqrt{d} = 2$$

Cons. id1 = $\times 0$

```
int i = 0
```

```
while (i < n) {
```

if ($arr[i] \geq 1$ && $arr[i] \leq n$) {

```
int correct_idx = ar[i] - 1;
```

```
if (ar[correct_idx] != ar[i]) {
```

```
swap(ar[correct_idx], ar[i]);
```

```
else { i++; }
```

else

 $i \rightarrow +;$

```
for (i=0; i<n; i++) {
```

```
if (ar[i] != l+1) return i+1;
```

```

    }
    return  $n+1$ ;
}

```

$$TC: O(n)$$

Sc: $O(1)$

$$ar[] = \left\{ \begin{array}{cccccc} & 1 & & & & \\ \cancel{3} & 2 & 3 & & & \\ \cancel{2} & \cancel{3} & \cancel{1} & 2 & -10 & 7 \\ 0 & 1 & 2 & 3 & 4 & 5 \end{array} \right\}$$

↑